

Aufgabe 1: Monitorkonzept (20 Punkte)

In der Zauberschule Hogwarts verwenden die Schüler der vier Häuser regelmäßig Zaubersprüche. Um Unfälle durch Zaubereien zu vermeiden hat Albus Dumbledore einen Synchronisationszauber über die Schule gelegt. Er bewirkt, dass immer nur ein Spruch ausgeführt wird. Alle anderen ausgesprochenen Sprüche werden in der Ausführung zurückgestellt.

Wenn mehrere Zaubersprüche zurückgestellt werden, dann werden zuerst die Sprüche des Hauses Huffelpuff ausgeführt, danach die Sprüche der Schüler von Ravenclaw, darauf folgenden die Sprüche der Slytherins und am Ende werden die Sprüche des Hauses Gryffindor ausgeführt.

Erstellen Sie zunächst den Synchronisationsgraphen.

Erstellen Sie zu diesem Szenario eine Lösung mit Hilfe des Monitorkonzeptes und der 3. Signalvariante.

```
monitor Hogwarts {

    private final int Gryffindor = 0;
    private final int Slytherin = 1;
    private final int Ravenclaw = 2;
    private final int Huffelpuff = 3;

    // ihre Objektdeklarationen

    public Hogwarts() {

    }

    entry jinxSpoken(int haus) {

    } // jinxSpoken

    entry jinxDone() {

    } // jinxDone

} // monitor
```

Aufgabe 2: Semaphorekonzept (20 Punkte)

Erstellen Sie mit Hilfe von Semaphoren einen Stack (Kellerspeicher). Dieser führt die Operationen push() und pop() aus. Es darf dabei zu keinen Über- oder Unterläufen kommen.

Erstelle Sie zunächst den Synchronisationsgraphen.

```
public Stack {  
  
    Object[] stack;  
    int topOfStack = 0;  
  
    // ihre Deklarationen  
  
    public Stack() {  
  
    }  
  
    public void push(Object el) {  
  
    } // push  
  
    public Object pop() {  
  
    } // pop  
  
} // Stack
```

Aufgabe 3: Prozessverwaltung (20 Punkte)

In Ihrem System sind 6 Prozesse aktiv. Die Prozesse 0,1 und 5 haben eine hohe Priorität und werden in der Ready Queue 1 verwaltet. Die Prozesse 2,3 und 4 haben geringere Priorität und werden in der Ready Queue 2 verwaltet. Das System verwendet ein verdrängendes Multilevel-Feedback Scheduling. Innerhalb der Ready Queues wird das Round Robin Scheduling Verfahren verwendet.

Wird ein Prozess mit höherer Priorität lafbereit, wird er automatisch zum Running Process und der abgelöste Prozess wird an den Anfang seiner Ready Queue gestellt.

Die Operation write Disk stellt den Auftrag in die Warteschlange der Festplatte. Nach Beendigung des Auftrages wird der DiskInterruptHandler aufgerufen, der den blockierten Prozess wieder freigibt und weitere anstehende Aufträge bearbeitet.

R_p steht für running process.

Zustand der Prozessverwaltung					
R_p	Ready Queue 1	Ready Queue 2	empty		Disk Queue
			Counter	Queue	
0	1,5	3,4,2	0	-	-
empty.v()					
write Disk()					
empty.p()					
timerItrHandler					
empty.p()					
timerItrHandler					
write Disk()					
DiskItrHandler					

Aufgabe 4: Speicherverwaltung

20 Punkte

- a) Die Hauptspeicherverwaltung teilt Speicherbereiche variabler Länge zu und führt über die freien Bereiche in der Freispeicherliste Buch. Der Listenkopf free benennt die Speicheradresse des Listenanfangs. In jedem freien Block stehen dessen Länge sowie die Adresse des nächsten freien Blocks. Zu Beginn sei in der Freispeicherliste der gesamte Speicher als zusammenhängender Bereich der Länge 100.

Freigegebene Speicherbereiche werden ggf. an das Ende der Freispeicherliste angehängt. Reste, die bei einer Anforderung übrig sind, bleiben in der Freispeicherliste an der Stelle des ausgewählten Speicherbereichs.

Falls eine Anforderung erfüllt ist, skizzieren Sie bitte jeweils die Speicherbelegung sowie die Freispeicherliste (free) nach deren Bearbeitung. Anderenfalls geben Sie bitte „nicht erfüllbar“ an.

- a) Es wird die First-Fit-Strategie verwendet und die folgenden Speicheranforderungen und –Freigaben treffen ein:

1. A1: anfordern (20)
2. A2: anfordern (20)
3. A3: anfordern (30)
4. freigeben der Anforderung A1
5. A4: anfordern (10)
6. freigeben der Anforderung A3
7. A5: anfordern (50)

- b) Es wird die Best-Fit-Strategie verwendet und die folgenden Speicheranforderungen und –Freigaben treffen ein:

1. A1: anfordern (20)
2. A2: anfordern (20)
3. A3: anfordern (30)
4. freigeben der Anforderung A1
5. A4: anfordern (10)
6. freigeben der Anforderung A3
7. A5: anfordern (50)

- b) Sie haben in Ihrem System einen Hauptspeicher mit den Kacheln K1 und K2. Im folgenden werden verschiedene Seitenanforderungen vorgenommen. Stellen sie die Anforderungen und deren Bearbeitungen nach der Least Recently Used und der Optimal Replacement Strategie dar. Wieviele Seitenfehler treten bei den Methoden auf?

<i>Least Recently Used</i>		
<i>Zugriff</i>	<i>K1</i>	<i>K2</i>
	-	-
1		
2		
3		
1		
2		
4		
1		
2		
3		
1		
2		
4		

<i>Optimal Replacement</i>		
<i>Zugriff</i>	<i>K1</i>	<i>K2</i>
	-	-
1		
2		
3		
1		
2		
4		
1		
2		
3		
1		
2		
4		

- c) Welche Prozesse laufen ab, wenn ein Prozess auf eine Variable zugreift, die auf einer Seite liegt, die sich gerade nicht im Hauptspeicher befindet?

Aufgabe 5: Dateisystem – Dateischutz in Unix 20 Punkte

a)

Im folgenden Unix-System gibt es 3 Benutzer A, B und C. Stellen Sie den Dateischutz so ein, dass die folgenden Aufrufe ohne Zugriffsrechtsverletzungen möglich sind:

- (1) A soll das Programm *p1* ausführen können.
- (2) B soll das Programm *p1* ausführen können.
- (3) C soll das Programm *p2* ausführen können.

Dabei gilt, dass Programm *p1* die Programme *p2* und *p3* aufruft. Ferner liest *p2* Daten aus der Datei *d1* und schreibt in *d2*. Programm *p3* liest aus *d2* und schreibt in *d1*.

Die Benutzergruppen *ABC* (Benutzer A, B und C), *BC* (Benutzer B und C) und *AB* (Benutzer A und B) sind definiert.

Dateibesitzer und Gruppenzugehörigkeit können Sie folgender Tabelle entnehmen.

Rechte	Besitzer	Gruppe	Datei
	B	ABC	d1
	C	BC	d2
	B	AB	p1
	C	BC	p2
	B	BC	p3

Tragen Sie in die Tabelle die notwendigen Zugriffsrechte-Definitionen ein. Vergeben Sie dabei nicht mehr Rechte als unbedingt notwendig sind (=> Prinzip der minimalen Rechte).

b) In welcher Datenstruktur werden bei UNIX die Zugriffsrechte gespeichert?

c) Wie erreicht UNIX, dass es sowohl kleine als auch große Dateien mit einem Dateideskriptor fester Länge effizient verwalten kann?